

■ Lab 15 — Advanced Terraform & CI/CD (AWS CloudShell)

■ Objectifs du Lab

- Utiliser le **dependency lock file**, le bloc `moved`, le bloc `check`
- Gérer les **secrets** de manière sécurisée
- Utiliser `terraform plan -out=planfile + terraform apply planfile`
- Mettre en place un pipeline **CI/CD GitHub Actions**

■ Étape 1 — Préparer l'environnement

```
bash $HOME/install-terraform.sh
mkdir -p $HOME/terraform-training/lab15-advanced
cd $HOME/terraform-training/lab15-advanced
```

Le reste des instructions Terraform est identique à la version principale (README.md).

■ Étape 2 — Créer le .gitignore AVANT tout

■ ****Critique — à faire avant tout `git add`.****

```
cat > .gitignore << 'EOF'
.terraform/
tfplan
tfplan-destroy
*.tfplan
terraform.tfstate
terraform.tfstate.backup
*.auto.tfvars
EOF
```

■ Étape 3 — Cloner le repo GitHub

```
git config --global user.email "<votre-email>"
git config --global user.name "<votre-prenom>"

git clone https://<votre-token>@github.com/<votre-compte>/tf-training-<votre-prenom>.git
cd tf-training-<votre-prenom>
```

■ Étape 4 — Créer le workflow à la racine du repo

■ ■ Le dossier `.github/workflows/` doit être à la ****racine du repo****.

```
# Se placer à la racine du repo cloné
cd $HOME/terraform-training/tf-training-<votre-prenom>

mkdir -p .github/workflows

cat > .github/workflows/terraform.yml << 'EOF'
name: Terraform CI/CD

on:
  push:
    branches: [ main ]
    paths: [ 'labs/lab15-advanced/**' ]
  pull_request:
    branches: [ main ]
    paths: [ 'labs/lab15-advanced/**' ]
  workflow_dispatch:
    inputs:
      action:
        description: 'Action à exécuter'
        required: true
        default: 'plan'
        type: choice
        options: [ plan, apply, destroy ]

env:
  TF_WORKING_DIR: labs/lab15-advanced
  TF_VAR_username: ${ github.actor }

jobs:
  terraform:
```

```

name: Terraform ${ github.event.inputs.action || 'plan' }
runs-on: ubuntu-latest
defaults:
  run:
    working-directory: ${ env.TF_WORKING_DIR }
steps:
  - name: Checkout code
    uses: actions/checkout@v4
  - name: Setup Terraform
    uses: hashicorp/setup-terraform@v3
    with:
      terraform_version: "1.15.5"
  - name: Configure AWS credentials
    uses: aws-actions/configure-aws-credentials@v4
    with:
      aws-access-key-id: ${ secrets.AWS_ACCESS_KEY_ID }
      aws-secret-access-key: ${ secrets.AWS_SECRET_ACCESS_KEY }
      aws-region: eu-west-1
  - name: Terraform Init
    run: terraform init
  - name: Terraform Format Check
    run: terraform fmt -check -recursive
  - name: Terraform Validate
    run: terraform validate
  - name: Terraform Plan
    run: terraform plan -var="environment=dev" -out=tfplan
    if: github.event_name == 'pull_request' || github.event.inputs.action == 'plan' || github.event_
  - name: Upload Plan
    uses: actions/upload-artifact@v4
    with:

    name: tfplan
    path: ${ env.TF_WORKING_DIR }/tfplan
    retention-days: 1
    if: github.event_name == 'pull_request' || github.event.inputs.action == 'plan' || github.event_
  - name: Terraform Apply
    run: terraform apply tfplan
    if: github.event.inputs.action == 'apply'
  - name: Terraform Destroy
    run: |
      terraform plan -destroy -var="environment=dev" -out=tfplan-destroy
      terraform apply tfplan-destroy
    if: github.event.inputs.action == 'destroy'
EOF

```

■ Étape 5 — Vérifier et commiter

```
# Vérifier ce qui sera commité
git status

# ■ Attendu : .github/, labs/lab15-advanced/*.tf, .terraform.lock.hcl
# ■ NE PAS voir : .terraform/, tfplan

git add .
git commit -m "feat: lab15 advanced terraform + CI/CD"
git push origin main
```

■ Validation du Lab

#	Critère	Validé
1	`gitignore` créé avant le premier `git add`	■
2	`.terraform/` et `tfplan` absents du repo	■
3	`.github/workflows/` à la racine du repo	■
4	Pipeline GitHub Actions se déclenche sur push	■
5	Apply et Destroy via `workflow_dispatch` fonctionnent	■

■ ****Fin du Lab 15 !****